1º DAM

Programación

Jesús Castellano González

Trimestre 1

Programación Estructurada

Anatomía de un Programa Informático

Unidad Didáctica 1

1.- Algoritmos y Programas

Introducción

Proceso de Resolución de Problemas por medio de Computadoras:

Fase 1: Resolución del Problema

- Producto resultante: Algoritmo.

Fase 2: Implementación en la Computadora

- Producto resultante: Programa (El algoritmo se expresa en un lenguaje de programación).

1.- Algoritmos y Programas

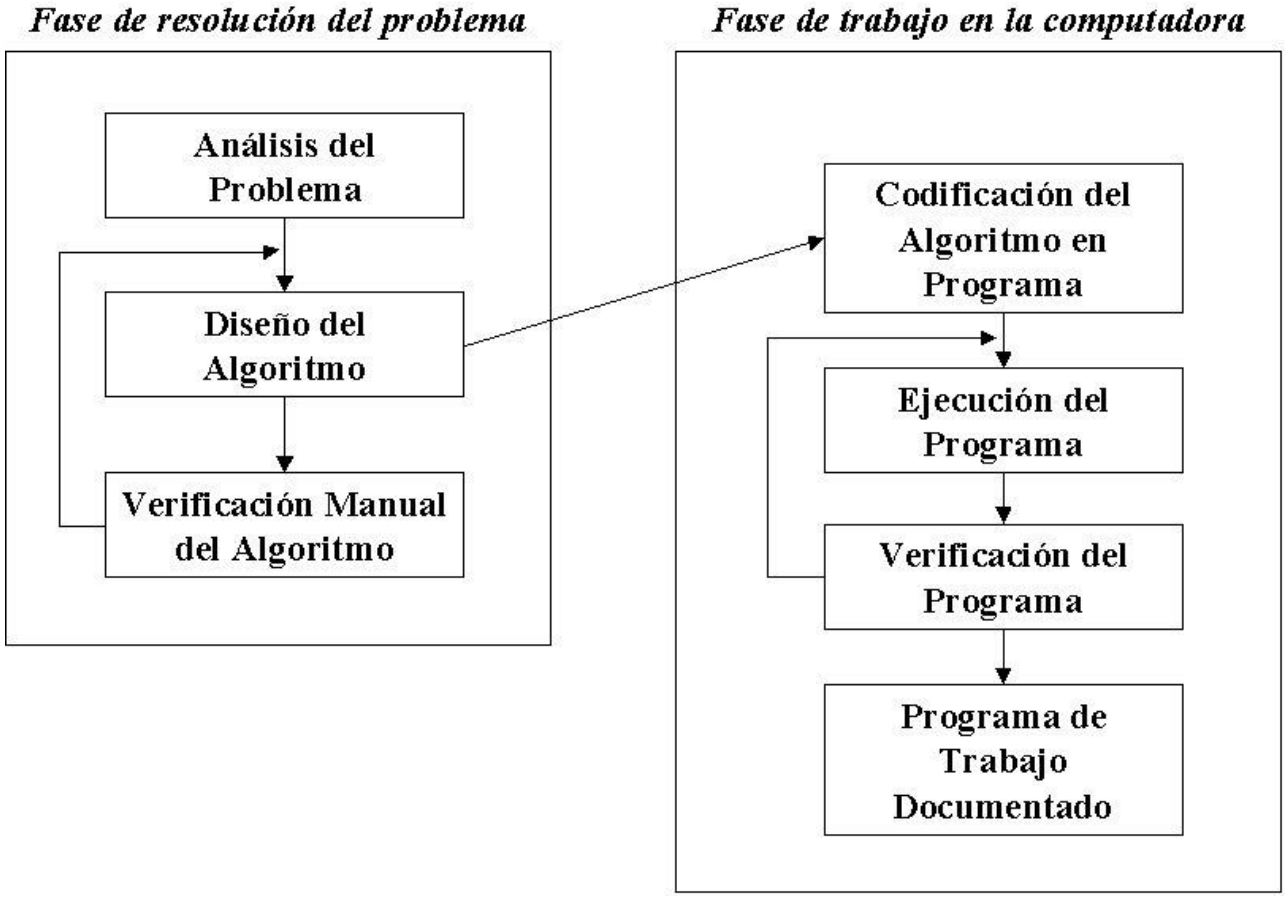
Definición de Algoritmo

"Conjunto de instrucciones que especifican la secuencia ordenada de operaciones a realizar para resolver un problema".

En otras palabras, un algoritmo es un método o fórmula para la resolución de un problema.

Ejemplo: Receta de cocina: Elaboración de una Tortilla.

Proceso de Diseño de Programas



Análisis del Problema

Primer paso para encontrar la solución a un problema.

Fases o Tareas:

- Definición lo más exacta posible del problema.
- Identificar los elementos de información dados en el problema que puedan ser útiles para obtener la solución (Datos de Entrada).
- Identificar qué tipo de información se necesita producir (Datos de Salida).



Análisis del Problema

Ejemplo 1

Una persona ha trabajado 38 horas en una semana dada y gana a razón de 7 € la hora. La tasa de impuestos del estado es del 15.5 % de su paga bruta. Se desea saber cuál es la paga bruta, el descuento por impuesto y la paga neta del trabajador.

Con estos datos se debe obtener toda la información solicitada. En resumen, se tiene:

Entrada	Salida
Horas trabajadas: 38	Paga bruta
Impuestos: 15.5 %	Total impuestos a pagar
Paga/hora: 7 €	Paga neta

Análisis del Problema

Ejemplo 2

Escribir un algoritmo que calcule la superficie de un rectángulo.

Este problema no está bien definido ya que las especificaciones de entrada y salida no han sido descritas.

Especificaciones de Entrada:

- ¿Qué datos son entrada?
- ¿Cuántos datos se introducirán?
- ¿Cuántos son datos de entrada válidos?

Especificaciones de Salida:

- ¿Cuáles son los datos de salida?
- ¿Cuántos datos de salida se producirán?
- ¿Qué precisión tendrán los resultados?
- ¿Se debe imprimir cabecera?

Diseño y Verificación del Algoritmo

Solución a problemas complejos: *"Divide y Vencerás"*.

Dividir el problema en subproblemas más fáciles de resolver que el principal.

Ejemplo 2: Cálculo de la Superficie del Rectángulo.

División en Subproblemas:

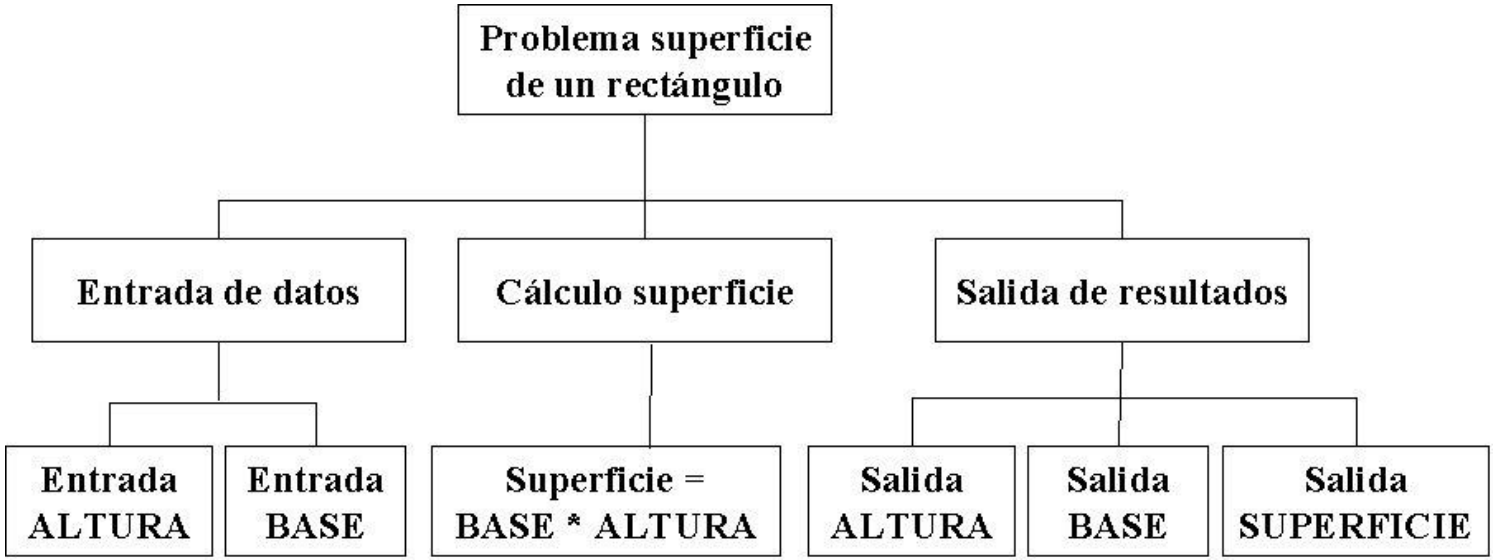
1. Entrada de datos.
2. Cálculo de la superficie (proceso).
3. Salida de resultados.

Diseño Descendente (Top-Down)

Diseño y Verificación del Algoritmo

Refinamiento Paso a Paso: Proceso de la rotura de un problema principal en etapas o subproblemas más sencillos. Pueden realizarse varios niveles de Refinamiento según la complejidad de los subproblemas.

Ejemplo 2: Cálculo de la Superficie del Rectángulo.



Diseño y Verificación del Algoritmo

Algoritmo: Conjunto de instrucciones que especifican la secuencia ordenada de operaciones a realizar para resolver un problema.

Características Generales:

- Debe ser **Comprensible** y **Preciso** (sin ambigüedades), e indicar el orden de realización de cada paso.
- Debe ser **Predecible (Definido)**. Si se aplica partiendo de la misma situación inicial, debe obtenerse siempre el mismo resultado.
- Debe ser **Finito**. El algoritmo debe terminar en algún momento (debe tener un número finito de pasos).

Diseño y Verificación del Algoritmo

Características Aconsejables:

- **Validez**: Un algoritmo es válido si carece de errores. Un algoritmo puede resolver el problema y no ser válido ya que posee errores.
- **Eficiencia**: Un algoritmo es eficiente si se obtiene la solución al problema en poco tiempo.
- **Óptimo**: Un algoritmo es óptimo si es el más eficiente posible y no contiene errores.

Elementos o Partes de un Algoritmo:

- **Entrada**: Datos iniciales proporcionados al Algoritmo.
- **Proceso**: Cálculos o acciones para encontrar la solución del problema.
- **Salida**: Resultados finales o respuestas del Algoritmo.

Diseño y Verificación del Algoritmo

Ejemplo 2: Cálculo de la Superficie del Rectángulo.

Algoritmo superficie_rectángulo

Paso 1. Entrada desde periférico de entrada (teclado) de base y altura.

Paso 2. Calcular superficie = base x altura.

Paso 3. Salida de base, altura y superficie a periférico de salida (pantalla).

Diseño y Verificación del Algoritmo

Lenguaje Algorítmico: Lenguaje empleado para representar el diseño de algoritmos.

- *"Independiente de cualquier lenguaje de programación particular"*.
- Permita una traducción clara del algoritmo al programa.
- Todo algoritmo será traducible a cualquier lenguaje de programación.

El Lenguaje Algorítmico se representará mediante **Herramientas de Programación Normalizadas**.

Diseño y Verificación del Algoritmo

Ejemplo 3:

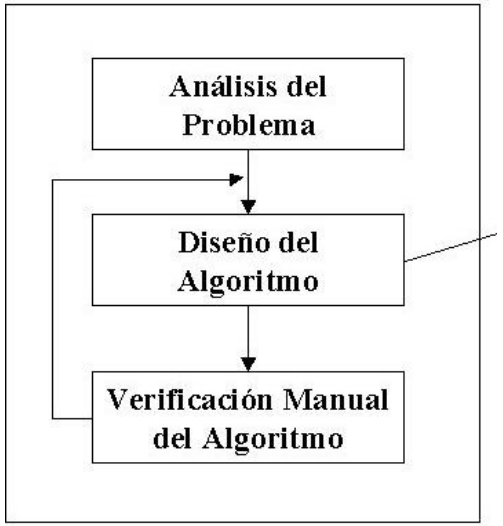
Un cliente ejecuta un pedido a una fábrica. La fábrica examina en su banco de datos la ficha del cliente, si el cliente es solvente entonces la empresa acepta el pedido, en caso contrario rechaza el pedido.

Los pasos del algoritmo son:

1. Inicio.
2. Leer el pedido.
3. Examinar la ficha del cliente.
4. Si el cliente es solvente aceptar pedido; en caso contrario rechazar pedido.
5. Fin.

Diseño y Verificación del Algoritmo

Verificación del Algoritmo



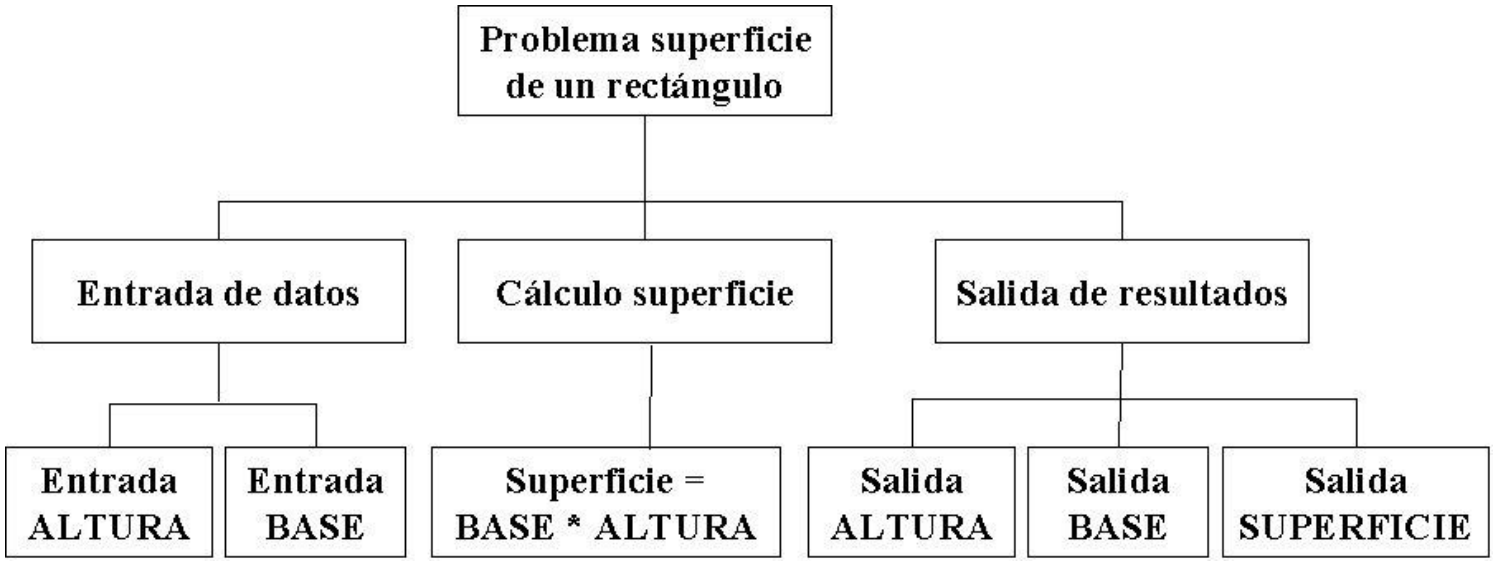
Prueba del Algoritmo: Ejecución manual usando datos significativos que abarquen todo el posible rango de valores y anotando los valores que van tomando en las diferentes fases, los datos de entrada o auxiliares y por último los valores de los resultados.

Herramientas de Programación

Diagrama de Estructura

La estructura de un algoritmo se puede representar en un diagrama en forma de bloques donde se muestren las diferentes tareas que deben ser ejecutadas y su relación entre ellas.

Ejemplo 2: Cálculo de la Superficie del Rectángulo.



Herramientas de Programación

Diagramas de Flujo (Flowchart)

Es una herramienta gráfica de descripción de algoritmos.

Características:

- Herramienta de programación por excelencia.
- Fáciles de diseñar porque el flujo lógico del algoritmo se muestra en un dibujo en lugar de con palabras.

Desventajas:

- Son difíciles de actualizar.
- Complejos de leer si ocupan varias páginas.

Herramientas de Programación

Pseudocódigo

Es una notación algorítmica textual.

Características:

- Lenguaje algorítmico similar al español o al inglés.
- Permite una redacción rápida del algoritmo.
- Es fácil de mantener.
- Fácil de convertir en programas escritos en lenguajes de programación específicos.

Desventaja:

- No es tan claro como un dibujo.

Diagramas de Flujo

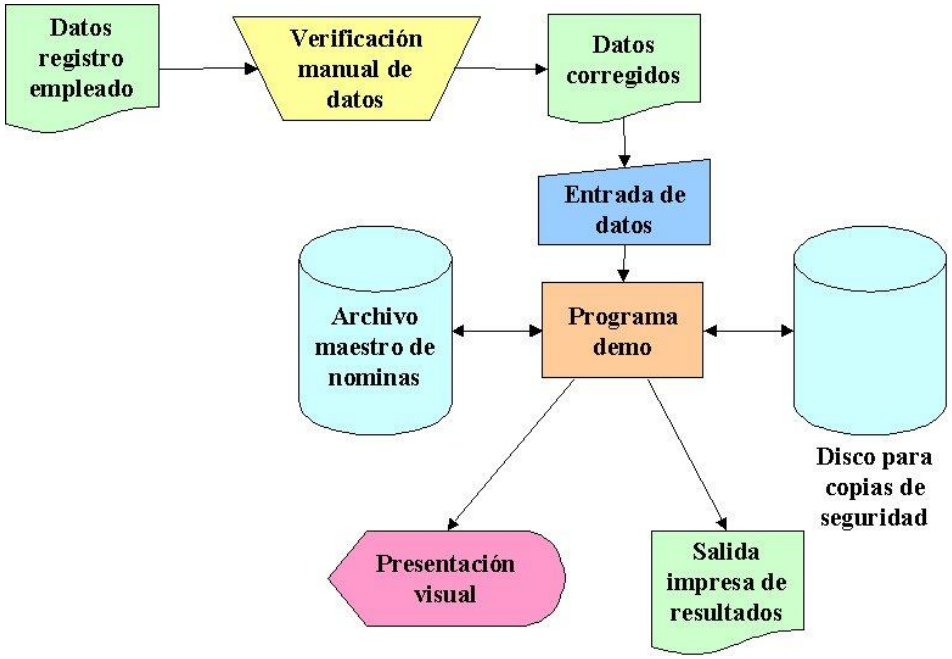
Diagrama que utiliza símbolos estándar y en el que cada paso del algoritmo se visualiza dentro del símbolo adecuado y el orden en que estos pasos se ejecutan se indican conectándolos con flechas llamadas **líneas de flujo** porque indican el flujo lógico del algoritmo.

Se caracteriza por utilizar un conjunto de símbolos gráficos normalizados y expresar de forma clara los posibles caminos de ejecución de las acciones (**secuencia**, **decisión** e **iteración**).

Diagramas de Flujo

Tipos de Diagramas de Flujo:

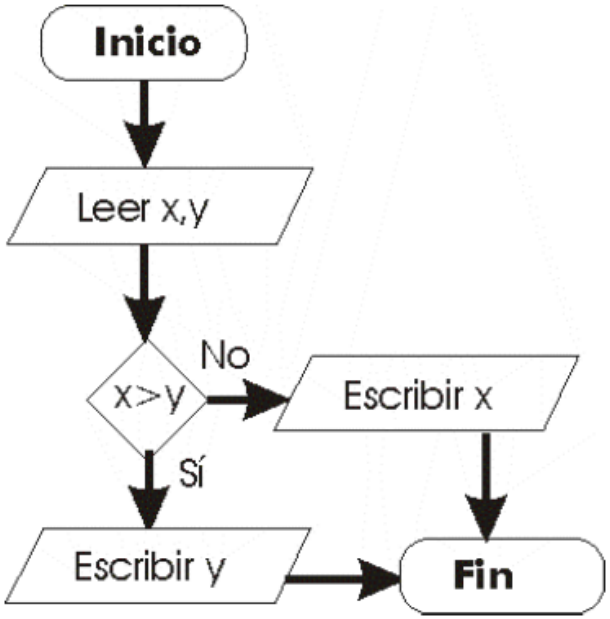
- **Diagrama de Flujo del Sistema (Organigramas)**:
  Describe las operaciones ejecutadas sobre los datos a través de todas las partes del sistema. El énfasis se pone en actividades, documentos, operaciones entre dispositivos del sistema, etc.



Diagramas de Flujo

Tipos de Diagramas de Flujo:

- **Diagrama de Flujo de Programas o de Detalle (Ordinogramas)**: Se limita a los pasos necesarios para describir un procedimiento particular: un algoritmo para resolver un problema mediante computadora.



Diagramas de Flujo

Notación

- **Entrada / Salida**: Este símbolo se utiliza siempre que se introduce una información a la computadora o sale información de la computadora.



Salida



Entrada

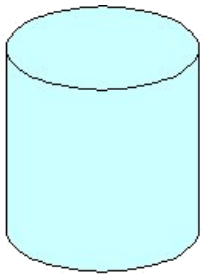
Para indicar un dispositivo de entrada o salida específico se podrá emplear:



Impresora



Teclado



Disco

Diagramas de Flujo

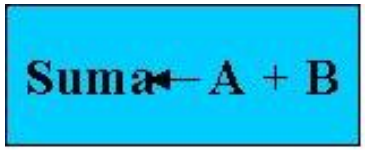
Notación

- **Proceso**: Representa la manipulación o procesamiento de los datos.

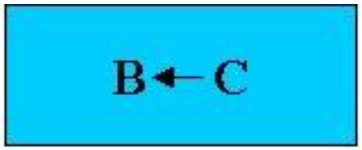


Tipos generales de operaciones de procesamiento:

- **Operaciones Aritméticas**.
- **Operaciones de Transferencia de Datos**.



Operaciones Aritméticas

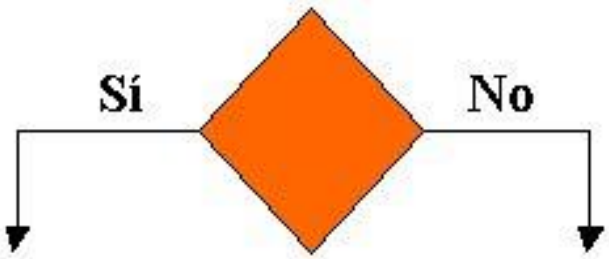


Operaciones Transferencia

Diagramas de Flujo

Notación

- **Decisión**: Representa operaciones en las que haya dos posibles alternativas a tomar frente a una decisión dada.



Diagramas de Flujo

Notación

- **Conectores**: Símbolos que se emplean para conectar partes lejanas de un diagrama de flujo con otras, sin utilizar líneas continuas o líneas que se crucen.

Tipos de Conectores:

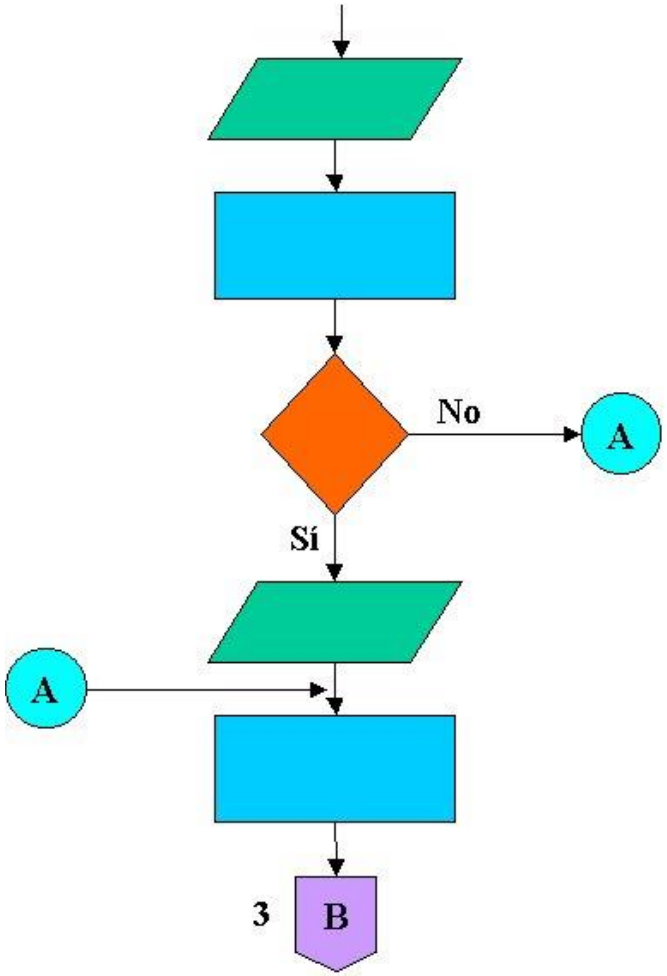
Conector: Permite enlazar el flujo del programa entre dos partes distantes del algoritmo que están dentro de la misma página.

Conector Externo: Permite enlazar el flujo del programa entre dos partes distantes del algoritmo que están en distintas páginas.

Diagramas de Flujo

Notación

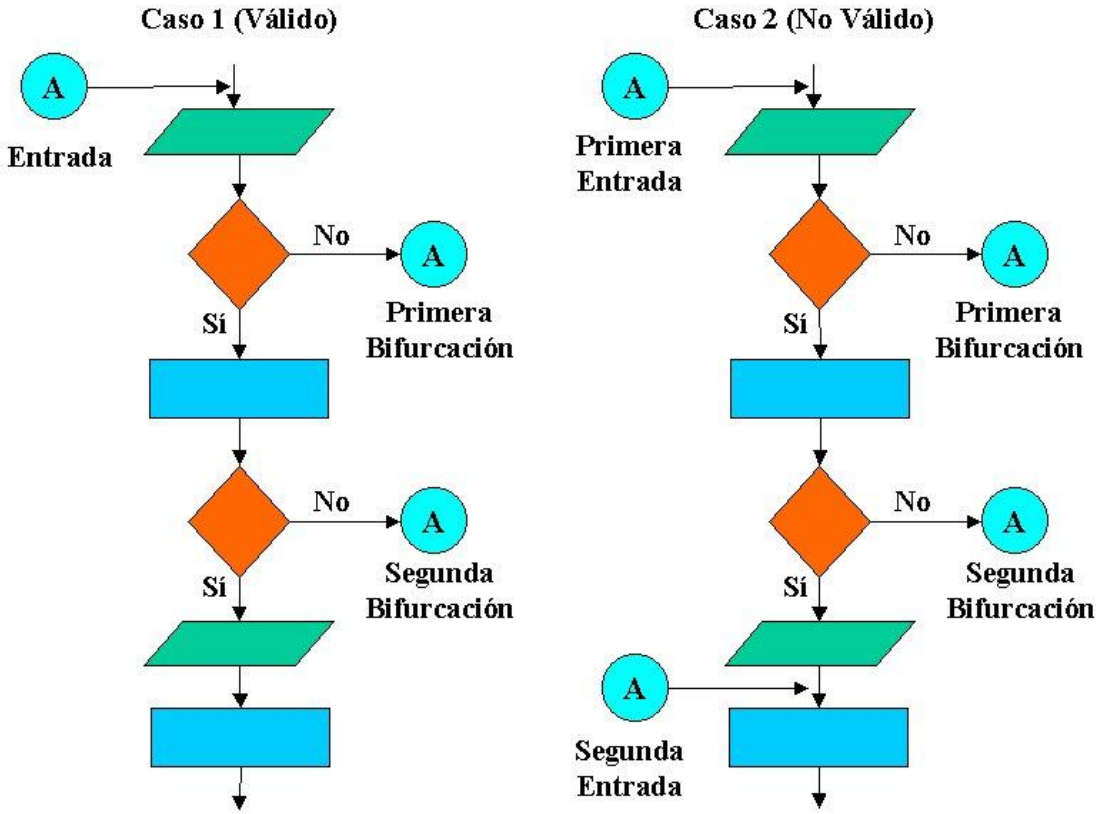
Tipos de Conectores:



Diagramas de Flujo

Notación

Validez del uso de Conectores:



Diagramas de Flujo

Notación

- **Terminales**: Se emplean para indicar el inicio y el fin de la ejecución de las instrucciones de un algoritmo.

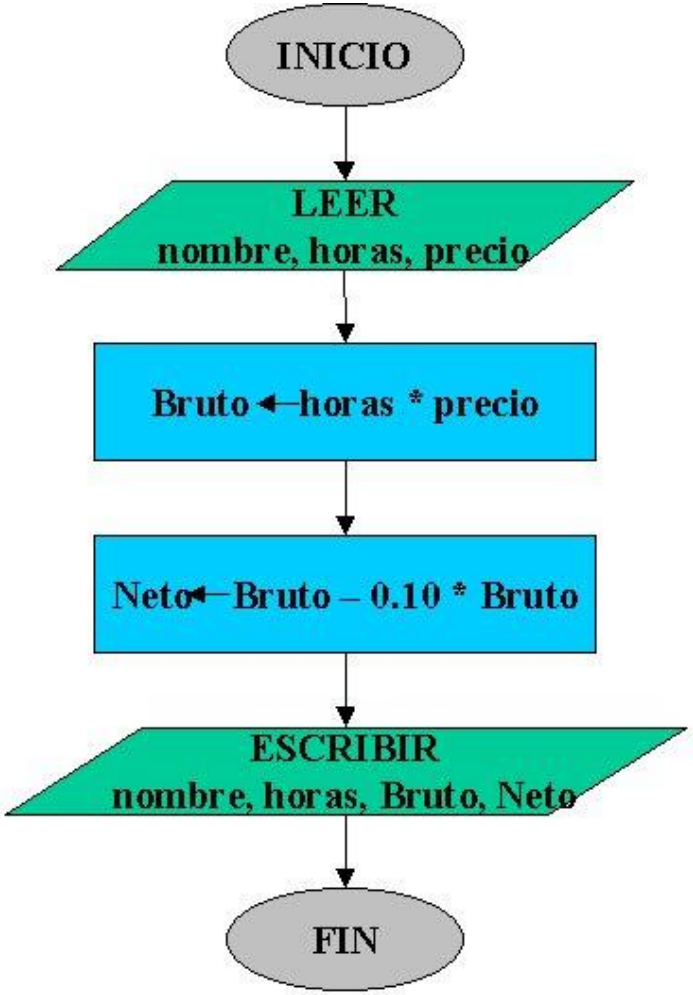




Diagramas de Flujo

Notación

Ejemplo: *El diagrama siguiente representa la resolución de un problema que deduce el salario neto de un trabajador a partir de la lectura del nombre, horas trabajadas, precio de la hora, y sabiendo que los impuestos aplicados son el 10 % sobre el salario bruto.*



Diagramas de Flujo

Notación

- **Subalgoritmos (Subprogramas)**: Representan a algoritmos o programas independientes que tienen entidad propia y pueden ser diseñados con independencia del programa principal.





Pseudocódigo

Es una mezcla de lenguaje natural y símbolos, términos y otras características comúnmente utilizadas en uno o más lenguajes de alto nivel.

A diferencia de los Diagramas de Flujo, permite la declaración de los datos (**constantes** y/o **variables**) manipulados por el algoritmo.

Dispone de un conjunto pequeño y completo de palabras reservadas que permiten expresar las acciones elementales que se realizan en el algoritmo.

Niklaus Wirdth (Bases de la Programación Estructurada): Cualquier problema algorítmico puede resolverse con el uso de tres tipos de instrucciones:

- Secuenciales
- Alternativas
- Iterativas

Pseudocódigo

Estructura de un Algoritmo

ALGORITMO <NombreAlgoritmo>

// Sección de Declaración de Constantes

// Sección de Declaración de Variables

// Comienzo de las acciones del Algoritmo

<Instruccion_1>

<Instruccion_2>

……..

FINALGORITMO // Fin del Algoritmo

Pseudocódigo

Concepto de Variable y Tipo de Datos

Variable: Almacén o contenedor de valores/datos que pueden cambiar durante la ejecución del algoritmo. Ocupan espacio en memoria y tienen asociado un identificador o nombre (que se utiliza en lugar de la posición de memoria) y un tipo (que determina su rango de valores y las operaciones permitidas).

Tipos de Datos Elementales:

- **NUMERICO**: Números tanto *Enteros* como *Reales*. Para separar decimales se utiliza el punto.
- **CADENA**: Cadenas de caracteres.
- **LOGICO**: Valores VERDADERO, FALSO.
- **ENTERO**: Números enteros (sin decimales).
- **REAL**: Números reales (con decimales).
- **CARACTER**: Letras, dígitos y símbolos empleados en la escritura.

Pseudocódigo

Declaración de Variables

Sintaxis:

DEFINIR <nombreVariable> **COMO** <TIPO>

DEFINIR <nombreVar1>, <nombreVar2> [,…] **COMO** <TIPO>

Ejemplos:

// Declara la variable X, de tipo entero

DEFINIR x **COMO ENTERO**

// Declara la variable dto, de tipo real

DEFINIR dto **COMO REAL**

Pseudocódigo

Declaración de Constantes

Constante: Almacén o contenedor de un valor/dato que no puede cambiar durante la ejecución del algoritmo.

Sintaxis:

<IDENTIFICADOR_CONSTANTE> ← <valor_literal>

Ejemplos:

```
PI <- 3.1416          // Declara la constante Pi
IVA <- 0.21           // Declara la constante Iva
```

Pseudocódigo

Operadores Aritméticos

Permiten la realización de operaciones matemáticas.

+	Suma
–	Resta
*****	Producto
/	División
MOD	Módulo (Resto de la división)
^	Exponenciación

Pseudocódigo

Prioridad de los Operadores Aritméticos

Indica el orden en el que serán evaluadas las expresiones aritméticas que contienen más de un operador. De mayor a menor:

^	Exponenciación
***, */, **MOD**	Producto, División, División Entera, Módulo
+, –	Suma, Resta

Ejemplos:

4 * 3 + 5

7 * 10 – 15 MOD 3 * 4 + 9

(7 * (10 – 5) MOD 3) * 4 + 9

Pseudocódigo

Operadores de Comparación

Realizan la comparación entre sus operandos devolviendo un valor booleano (Verdadero / Falso):

=	Igual
<>	Distinto
<=	Menor o Igual que
>=	Mayor o Igual que
<	Menor que
>	Mayor que

Pseudocódigo

Operadores Lógicos

Permiten la realización de relaciones lógicas (Verdadero / Falso) entre expresiones:

NOT	Negación	**NO**
AND	Conjunción	**&**
OR	Disyunción	**|**

p	NOT p
V	F
F	V

p	q	p AND q	p OR q
V	V	V	V
V	F	F	V
F	V	F	V
F	F	F	F

Pseudocódigo

Prioridad de los Operadores

Indica el orden en el que serán evaluadas las expresiones que contienen más de un operador. De mayor a menor:

^	Exponenciación
***, /, MOD**	Producto, División, División Entera, Módulo
+, –	Suma, Resta
=, <>, <, >, <=, >=	Operadores de Comparación
NO	Negación
&, |	Conjunción, Disyunción

Pseudocódigo

Instrucciones Elementales: Asignación

Sintaxis:

<variable> ← <expresion>

Comentarios:

- La variable situada a la izquierda del símbolo ← recibe el valor de la expresión de la derecha.
- <expresion> puede ser, un valor literal, una variable o el resultado de una operación entre variables y literales.

Ejemplos:

```
superficie ← base * altura        // Superficie rectángulo
impuestos ← 0,19                  // Establece el % de impuestos
x ← y + 3
```

Pseudocódigo

Instrucciones Elementales: Lectura

Sintaxis:

LEER <variable>

Comentarios:

- Guardar el valor leído de la entrada en la variable que se indica.
- El argumento de la acción LEER siempre debe ser una variable.
- Pueden leerse varios datos, sin más que poner una lista de variables separados por comas.

Ejemplos:

LEER x // Almacena el valor leído en la variable x

LEER base, altura // Lee dos datos, valor de la base y altura

Pseudocódigo

Instrucciones Elementales: Escritura

Sintaxis:

ESCRIBIR <expresión>

Comentarios:

- Escribir en la salida el valor resultante de evaluar la expresión que se indica.
- <expresión> puede ser, un valor literal, una variable o el resultado de una operación entre variables y literales.
- Pueden escribirse varios datos, sin más que poner una lista de expresiones separadas por comas.

Ejemplos:

ESCRIBIR "Esto es un mensaje de texto"

ESCRIBIR x

ESCRIBIR base, altura, base * altura

Pseudocódigo

Estructuras de Control: Secuenciales

Sintaxis: Secuencia de Instrucciones:

<Instruccion_1>

<Instruccion_2>

...

<Instruccion_n>

Ejemplo:

LEER x

LEER y

sum ← x + y

ESCRIBIR sum

